



第八讲 生成模型 Lecture 8 Generative Models

明玉瑞 Yurui Ming
yrming@gmail.com

声明

Disclaimer

- 本讲义在准备过程中由于时间所限，所用材料来源并未规范标示引用来源。所引材料仅用于教学所用，作者无意侵犯原著者之知识产权，所引材料之知识产权均归原著者所有；若原著者介意之，请联系作者更正及删除。

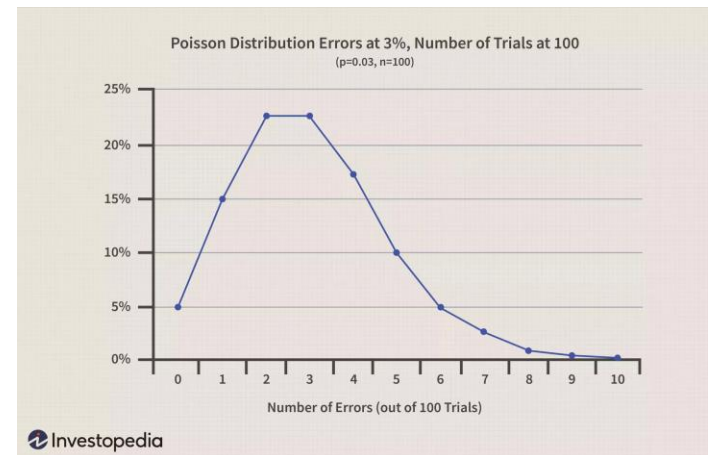
The time limit during the preparation of these slides incurs the situation that not all the sources of the used materials (texts or images) are properly referenced or clearly manifested. However, all materials in these slides are solely for teaching and the author is with no intention to infringe the copyright bestowed on the original authors or manufacturers. All credits go to corresponding IP holders. Please address the author for any concern for remedy including deletion.

采样问题

Sampling Problem

- 在实践中，我们经常需要从特定分布中采样，以进行研究或测试。比如，我们开发了一个客户服务系统，要对之进行压力测试。我们知道，一般，到店客户数量服从泊松分布。我们可从特定时段的泊松分布采样，来模拟系统需要服务的数目。

In practice, we have to sample from a specific probability distribution to cater to some requirement, such as for research or test. For example, to exert a load test on a newly-implemented customer serving system. We know usually the number of customers visiting a complex department is in accordance with Poisson distribution. So, we can sample from an instance of Poisson distribution on specified occasions to simulate the customers to test the system.



采样问题

Sampling Problem

- 什么是采样呢？说白了，就是生成特定分布的随机数或样本。比如，生成符合均匀分布的特定范围内的随机数，就是所谓的采样。

What is sampling? In plain text, it is to generate the numbers (or samples) that in accordance with some specific distribution. For instance, if it is desired to generate numbers of uniform distribution in a given range, it is the so called sampling.

- 这只是一个比较简单的例子，有时我们需要从更为复杂的分布采样，这时会有两个问题：

The above is just a simple example, sometimes, we have to sample from a much more complicated distribution,

- 这个分布很复杂，不好直接采样；

The distribution is overly complicated and rendering direct sampling very difficult;

- 基于领域知识，我们只对样本分布有一个大概的了解，无法写出解析表达。

We only have a gross understanding of the distribution; an analytic form of the distribution is intractable.

换元

Change of Variable

- 这个分布很复杂，不好直接采样，在这种情况下，会采用换元法，求出简单变换到复杂变换下的关系式；这样可以先从简单分布中采样，再变换到复杂分布下的样本。

If some distribution is overly complicated and prohibits direct sampling, in this circumstance, the change of variables will be adopted to find the relation which guides the shift from the simple distribution to complex distribution. In this way, we can sample from the simple distribution then convert it to the complicated distribution.

Because probabilities are nonnegative, and because the value of x must lie somewhere on the real axis, the probability density $p(x)$ must satisfy the two conditions

$$p(x) \geq 0 \quad (1.25)$$

$$\int_{-\infty}^{\infty} p(x) dx = 1. \quad (1.26)$$

Under a nonlinear change of variable, a probability density transforms differently from a simple function, due to the Jacobian factor. For instance, if we consider a change of variables $x = g(y)$, then a function $f(x)$ becomes $\tilde{f}(y) = f(g(y))$. Now consider a probability density $p_x(x)$ that corresponds to a density $p_y(y)$ with respect to the new variable y , where the suffices denote the fact that $p_x(x)$ and $p_y(y)$ are different densities. Observations falling in the range $(x, x + \delta x)$ will, for small values of δx , be transformed into the range $(y, y + \delta y)$ where $p_x(x)\delta x \simeq p_y(y)\delta y$, and hence

$$\begin{aligned} p_y(y) &= p_x(x) \left| \frac{dx}{dy} \right| \\ &= p_x(g(y)) |g'(y)|. \end{aligned} \quad (1.27)$$

One consequence of this property is that the concept of the maximum of a probability density is dependent on the choice of variable.

无解析形式密度函数问题

Intractability of PDF

- 对于第二种情况，即无法直接写出解析表达，比如，之前讲到人脸像素值的概率分布，我们知道的只是一个个样本，即生活中我们所见的人脸；并且，对于给定的一个样本，我们还指导究竟是不是人脸；至于这样的分布有怎样的一个解析表达，大概没有人知道。

For the second case, aka., no direct analytical form of probability mass function or probability density function, an example is we are only aware of individual faces during daily life with regard to the pixel value distribution of human face. And for any presented sample, we have the knowledge to decide whether it is human face or not. However, for a potential analytic form of sample distribution, no one knows.



对抗式生成网络

Generative Adversarial Networks

- 通用逼近定理：在人工神经网络的理论研究中，有一个很著名的定理，即如果网络足够大，足够深，则其可以以任意精度逼近任意性质良好的函数。假设我们知道复杂概率分布函数的解析形式，则我们可能可以求出来这个转换函数，问题是我們不知道。但既然神经网络可以逼近任意函数，则我们可以假设神经网络亦可以逼近这个转换函数，问题是，怎样衡量这个由神经网络建模的转换函数就是我们需要的。

Universal proximation theorem: during the theoretic research of neural networks, it came up the theorem that by leveraging sufficient deep and wide neural network, any complicated function with well-stated properties can be approximated in any precision. If we know the analytical form of the complicated PFD, then we can try to derive the transform function. The situation is we don't know. But considering arbitrary functions could be approximated by neural networks, we just assume neural networks can approach such pre-exist function by conjecture. The problem is, how can we measure the function modeled by the network is all we needed.

对抗式生成网络

Generative Adversarial Networks

- 要度量我们建模的转换函数的质量，能利用的只有一个个具体的样本；那么怎么利用这些样本呢？一个直观的想法是，假设用网络表征的转换函数记为生成器，或 $G(z)$ ，如果从这个函数中采出或生成的样本，与实际样本不可区分，则我们认为这个转换函数正是我们需要的。我们随后便可以从中采样，以便进行研究或测试。但人工判断这些样本的不可区分性，徒劳费神，因此我们可以再构造一个网络，称为判别器，或 $D(x)$ ，来达到整体的目的。

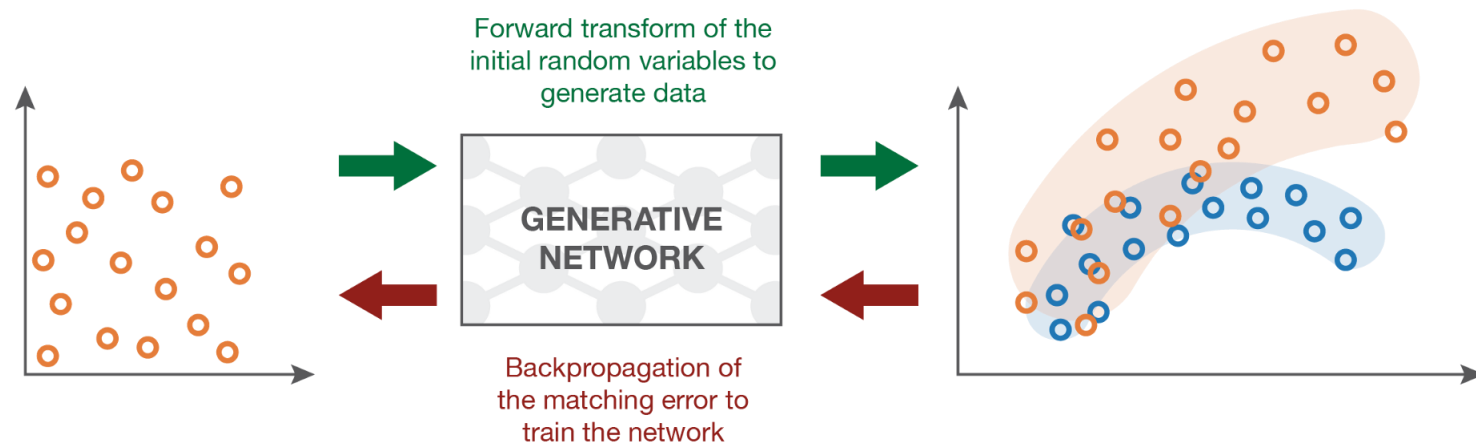
To measure the quality of the modeled transform function by utilizing the individual concrete samples, an intuitive idea is by harnessing the approximation capacity of networks, which denoted as generator or $G(z)$, to distinguish the samples from the generated and real samples. If they are indistinguishable, then such a $G(z)$ is the desired one. However, to manually identify these samples are time consuming, so, we can construct another network named discriminator or $D(x)$ for such a purpose.

对抗式生成网络

Generative Adversarial Networks

- ▶ 对抗式生成网络的想法如下图所示：

The idea for GAN is shown below:



Input random variables
(drawn from a uniform).

Generative network
to be trained.

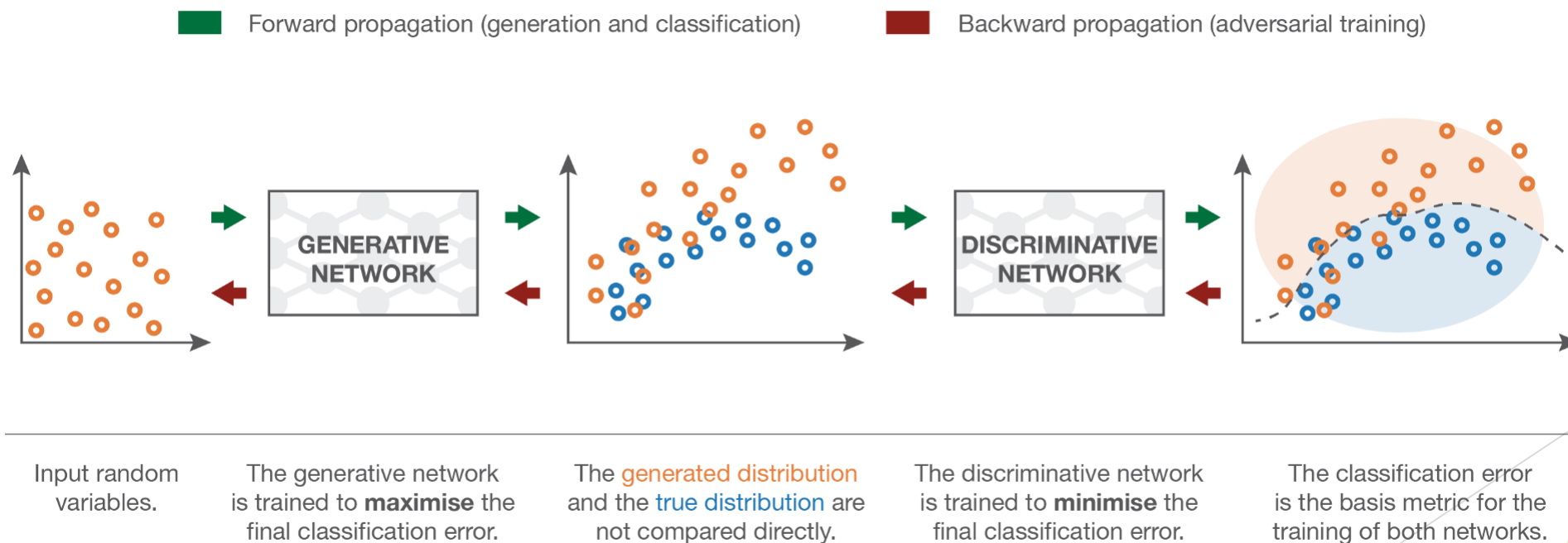
The **generated distribution** is compared
to the **true distribution** and the "matching error"
is backpropagated to train the network.

对抗式生成网络

Generative Adversarial Networks

► 对抗式生成网络的架构如下图所示：

The architecture for GAN is shown below:

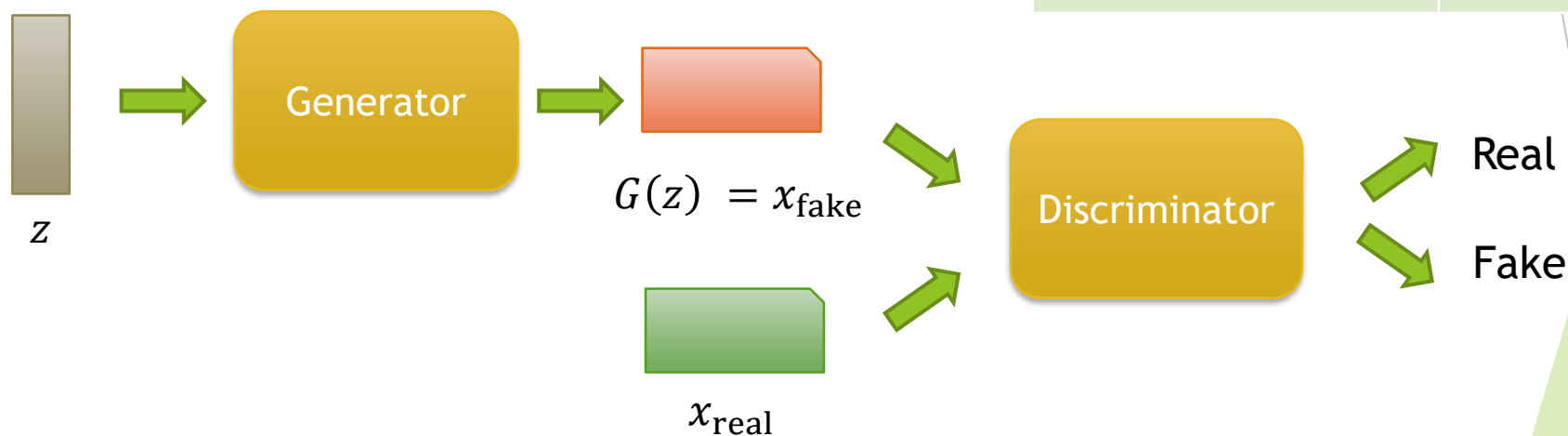


对抗式生成网络

Generative Adversarial Networks

- 对抗式生成网络的实现如下图所示：

The implementation for GAN is shown below:



符号 (Symbol)	意义 (Meaning)
z	噪声，可以认为从简单分布中的随机采样 noise, can be treated as samples from simple distributions

符号 (Symbol)	意义 (Meaning)	符号 (Symbol)	意义 (Meaning)
x_{real}	真实图片； real images	x_{fake}	生成图片； generated images
G	生成器； generator	D	判别器； discriminator
$G(z)$	生成器输出； generator output	$D(x)$	判别器输出； discriminator output

对抗式生成网络

Generative Adversarial Networks

- ▶ 显然，为达成所述之目的，就需要定义恰当的损失函数，以便可以用反向传播算法训练；对于判别器，我们期望其输出是判定为真实图片的概率，则可构造如下的损失函数：

Obviously, to fulfill the aforementioned aim, it is necessary to define the proper loss function in order to apply the back-propagation algorithm. For the discriminator, by design the output is interpreted as the probabilities of real images. So, the constructed loss function is as follows:

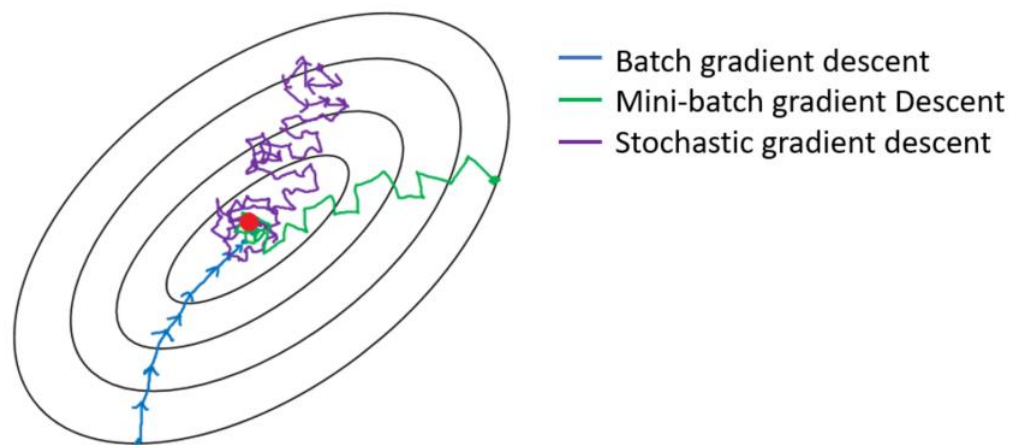
$$\max_D L(D) = 1 \cdot \log D(x_{\text{real}}) + 1 \cdot \log (1 - D(G(z)))$$

对抗式生成网络

Generative Adversarial Networks

- 注意上述损失函数只是为了便于理解，真实场景必须批量训练。原因在于，GAN要求的是生成样本的分布与真实分布不可区别。而只有当样本足够多的时候才能谈分布，单个样本是无法谈分布的。而前面无论CNN与RNN，单个样本是可以训练的，无非就是过程曲折些。

Please note the loss function above is just for illustrative understanding. In practice the training must be performed in the batch mode. The reason is for GAN, it only distinguishes the distributions of real samples v.s. generated samples. And the concept of distribution only holds for a collective of samples instead of a single one. But the CNN and RNN mentioned before can be trained with one sample in an inefficient and unstable process.



对抗式生成网络

Generative Adversarial Networks

- 实际可行的损失函数如下：

In practice, the loss function is as follows:

$$\max_D L(D) = \sum_{i=1}^m 1 \cdot \log D(x_{\text{real}}^{(i)}) + \sum_{j=1}^n 1 \cdot \log(1 - D(G(z^{(j)})))$$

- 或写成更为紧凑的形式：

Or re-write as a more compact form:

$$\max_D L(D) = \mathbb{E}_{x \sim p_x(x)} \log D(x) + \mathbb{E}_{z \sim p_z(x)} \log(1 - D(G(z)))$$

对抗式生成网络

Generative Adversarial Networks

- ▶ 对于生成器，我们期望其所生成之图片与真实图片尽可能接近，即被判为真实图片的概率尽可能高，则其损失函数为 $L(G) = 1 \cdot \log(D(G(z)))$ 尽可能大，由于先前已有 $1 \cdot \log(1 - D(G(z)))$ 形式，则要求 $L(G) = 1 \cdot \log(D(G(z)))$ 尽可能小。合在一起，便有下面minimax问题：

For the generator, we expect that the generated images are identical to the real images. Alternatively speaking, the chance of being categorized into real image class is the greater the better. Hence for the loss function $L(G) = 1 \cdot \log(D(G(z)))$, G should maximize its value. Since an alternative form $1 \cdot \log(1 - D(G(z)))$ has appeared previously, we can henceforth minimize this reciprocal term. Put it together, we have the following minimax game problem:

$$\min_G \max_D L(D, G) = \sum_{i=1}^m 1 \cdot \log D(x_{real}^{(i)}) + \sum_{j=1}^n 1 \cdot \log(1 - D(G(z^{(j)})))$$

对抗式生成网络

Generative Adversarial Networks

- 右图展示了训练 GAN 的一种实现:

The right figure shows a way for training GAN:

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

自编码器

Autoencoder

- ▶ 对于GAN来说，当训练完毕，生成器会生成接近真实样本的样本。一般，这些样本能为我们感知（知道是什么东西）。而还有利用神经网络的另一种模型，称为自编码器，当其训练完毕时，编码器生成的东西可能不能为我们直接理解，我们称为隐表征。这个隐字，是从概率统计里面的隐变量衍生过来的，隐变量指本身不能被直接观测的变量。因此，可以说隐表示是本身不能被直接理解的表示。对于解码器，其借助隐变量或隐表征来还原原来的表示。

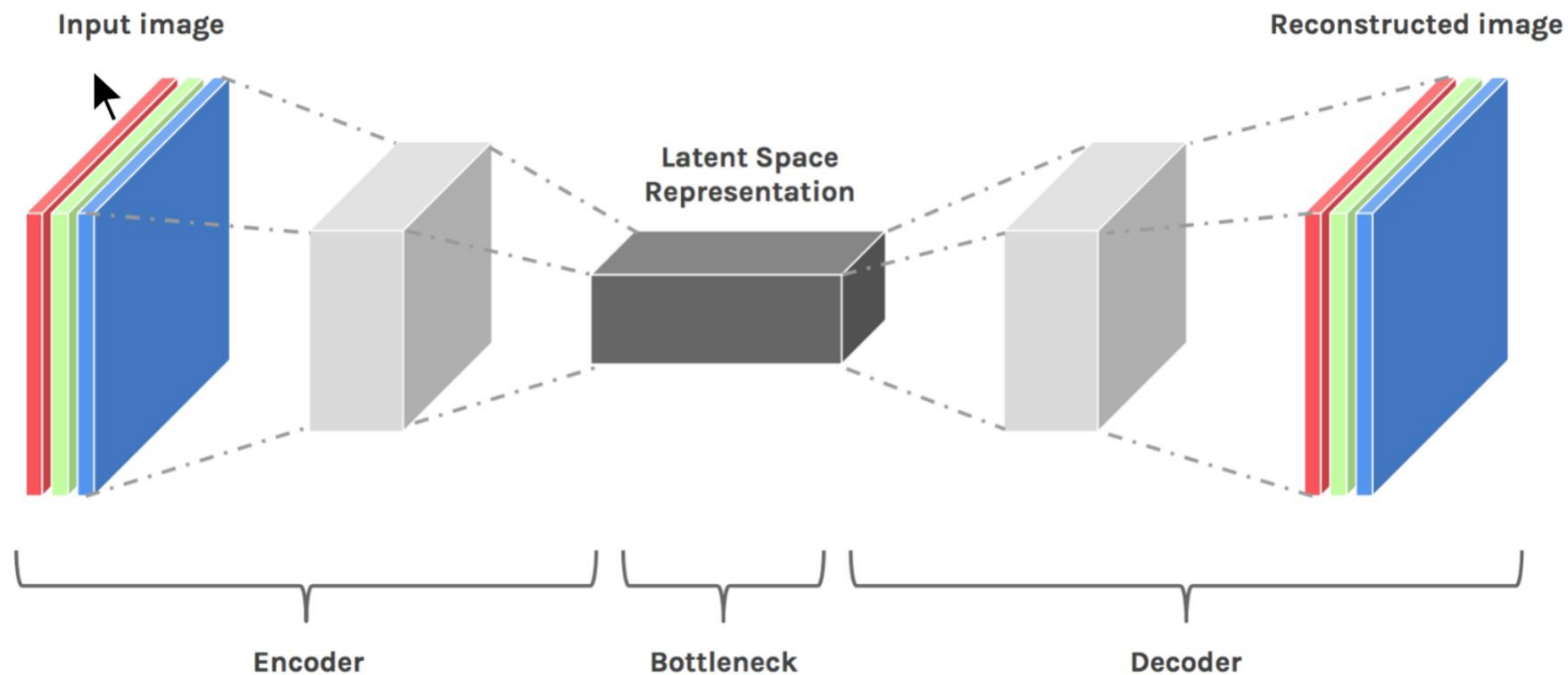
For GAN, upon finish of training, the generator will output samples resemble the real ones. Generally, these samples are conceivable (we understand these samples). But there also exists another generative model which utilizes neural network, called autoencoder. When training finishes, usually we cannot understand the output content from encoder, and we call it latent representation. The wording surrounding “latent” derives from the latent variables from probabilities and statistics. Latent variables themselves cannot be directly observed, which means the latent representations cannot be directly comprehended. For decoder, it tries to restore the original representation based on the latent variable or latent representation.

自编码器

Autoencoder

- 自编码器的架构如右图所示：

The right figure shows the general structure of an autoencoder:



自编码器

Autoencoder

- 如上图所示，自编码器通常由深度神经网络实现两部分组成。靠近输入端的模型被称为编码器，而靠近输出端的模型则被称为解码器。我们用 x 输入（通常是一幅图像），用 $\hat{x}$ 表示输出（通常是另一幅图像）。位于编码器和解码器中间的向量被称为隐变量，记作 z 。

As shown in the previous figure, autoencoder consists of a pair of models (often realized by deep neural networks). The one located near the input is called an encoder whereas the one located near the output is called a decoder. We denote the input (typically an image) as a vector x , and the output (typically another image) as a vector $\hat{x}$. The vector located in the middle between the encoder and the decoder is called a latent variable, denoted as z .

- 自编码器也可以用来降维，与PCA方法类似，既可以从代数上解释，又可以从概率模型上解释。所以，自编码器与概率模型也有一定的联系，因此，自编码器理论中一个重要的概念便是上面的隐变量。

Autoencoder can also be used for dimension reduction. Similar to PCA, one interpretation is from linear algebra perspective, but we can also do it from probabilistic model. Therefore, autoencoder also has some connections with probabilistic model so one important concept in autoencoder is latent variable.

自编码器

Autoencoder

- ▶ 在概率模型中，隐变量 z 是指我们无法观测到的变量。尽管其和模型本身相关，但它们不是训练数据集的一部分。

In a probabilistic model, latent variables z are variables that we do not observe and hence are not part of the training dataset, although they are part of the model.

- ▶ 编码器的任务是为 x 提取有意义的特征表示，而解码器的任务则是根据隐变量 z 生成一幅新的图像。因此，对于输入而言，隐变量 z 封装了可用于描述 x 的信息。编码过程可能是一个有损的过程，但我们的目标是尽可能多地保留 x 的重要内容。对于输出而言，隐变量充当了生成图像 $\hat{x}$ 的“种子”。在理论上，两个不同的 z 应该会为我们带来两幅不同的生成图像。

The job of the encoder is to extract a meaningful representation for x , whereas the job of the decoder is to generate a new image from the latent variable z . With respect to the input, the latent variable z encapsulates the information that can be used to describe x . The encoding procedure could be a lossy process, but our goal is to preserve the important content of x as much as we can. With respect to the output, the latent variable serves as the “seed” from which an image $\hat{x}$ can be generated. Two different z 's should in theory give us two different generated images.

自编码器

Autoencoder

- 注意，在某些应用中，我们可能仅关注编码器，此时，隐变量更习惯称为隐表示。为什么需要隐表示呢？比如，对一个图片来说，各部分可能是高度相关的，在某些应用中，可能希望这个图片有另外一种维数较小的紧表示。我们经过编码，寻求隐表示，然后再经过解码，与原来的数据进行比较再现原本的状态，进行优化编码表示以达到目的。

Notably, in some applications, we only care about encoder, and the latent variable is generally referred as latent representation. The motivation for constructing an auto-encoder can be for dimension reduction. Take an image for example, there might exist highly dependent regions among the picture, and some application might desire a low-dimensional compact representation. We first encode the original image to seek the required latent representation, then we decode it to compare with the original data in some fidelity, so as to optimize the encoder for demanded representation.

- 在原始的自编码器中，由于隐变量难以构造，解码器往往难以单独使用，因此需要引入变分自编码器，以便更加容易地构造隐变量从而进行有效生成。

To use decoder from naïve autoencoder in a standalone way is impractical, since there is no obvious way to construct latent variable. People later introduced variational autoencoder, in order to facilitate latent variable construction to effectively generate contents.

习题

Problems

1. 阅读如下链接的材料，尝试实作生成手写数字的GAN的应用。

Read the following material to implement a generative adversarial network to generate hand-written digits.

<https://www.oreilly.com/content/generative-adversarial-networks-for-beginners/>

2. 阅读如下链接的材料，尝试实作生成手写数字的隐表示。

Read the following material to implement an auto-encoder to generate the latent representation of hand-written digits.

<https://www.edureka.co/blog/autoencoders-tutorial/>